Logs, automation and test tools reference

Use with the guide's event collection, denial investigation and capacity automation for FWR-07/08 and FWT-07/08. J approves event/log work. The explicit L automation portion covers the script, target, private paths, validation and cleanup. Physical HP runs OPNsense. Dell hosts DC1, WIN1 and APP1 only.

Record actual builds, named lab sources/targets, synthetic identities, single action/rate, expected results, evidence, stop condition and rollback before execution. The applicable D/F/H/I/K approvals must cover any active mock dependency. Use the private map, Windows auditing, APP1 host policy and Apache roles. Never target home/production.

DC1 Security log and auditing

Target / approval: DC1, approved event reader; administrator only for separately approved audit corrections. J permits observation; policy changes need the named E/F scope. Have console recovery and the current audit-policy record.

1. On DC1 open **Server Manager > Tools > Group Policy Management**. Inspect the GPO linked to **Domain Controllers**: **Computer Configuration > Policies > Windows Settings > Security Settings > Advanced Audit Policy Configuration > Audit Policies > Account Logon > Audit Kerberos Authentication Service**, success and failure. Compare its winning/effective policy on DC1. A WIN1 OU policy does not apply to DC1; do not change or clear auditing merely to obtain a record.
2. Open **Event Viewer > Windows Logs > Security > Filter Current Log**. Set the approved narrow time range and IDs **4768,4771**. Event 4768 describes a Kerberos ticket-granting-ticket request on a domain controller; verify its success/result fields. Event 4771 records pre-authentication failure, commonly a wrong password. A WIN1 interactive/cached logon is not automatically a DC1 authentication event.
3. Inspect matching account, client address, result/failure code, record ID and **Details > XML View > TimeCreated SystemTime**. Verify actual event version/build; do not infer success from the ID alone. Compare with the action and clock record below.
4. Inspect **Security > Properties**: maximum size, retention/overwrite behavior and approved readers. Compare with the measured disk/retention plan and existing approved access. Keep raw event exports private. Missing rights or audit coverage blocks evidence; request only the bounded correction, never clear the log.

Microsoft's 4768 and 4771 references define these fields. Updated Windows builds can change the 4768 layout. Do not disable Kerberos pre-authentication or lockout to manufacture E02.

OPNsense Live View and effective policy

Target / approval: dedicated lab administrator browser to physical HP OPNsense; J read scope and existing D/K action/rule approval. Preserve the approved management path and HP console.

1. Open **Firewall > Log Files > Live View**. Filter the exact lab ingress interface, source, destination, protocol/port and time; use AND conditions. Confirm the intended deny has logging enabled under its existing approval. Select the event's detail/info/rule link and retain the rule identifier privately.

2. Compare the saved rule, source/destination aliases and effective order with the event. Inspect floating/group/automatic and interface rules, quick behavior, translations and **Firewall > Diagnostics > States**. A stateful flow may log its first packet only; a repeated request need not create another entry. Do not flush all states or broaden access.
3. Confirm the target is listening and a permitted client succeeds for that same service. For an unlisted Corporate-to-Restricted test, reuse the existing approved DC1-to-APP1 TCP 443 request with WIN1's working HTTPS control. Match the actual OPNsense block; a later host drop or timeout does not identify the firewall rule.
4. Inspect **System > Settings > Logging** and the selected release's local retention/access settings against the space budget. Retain the event-time policy/configuration reference: a rule link can describe a later changed rule. No collector, packet capture or remote syslog service is added.

See OPNsense firewall logging and rule processing. If the view is empty, check logging, filters, clock and states before any further action. Preserve the gap as NOT TESTED; do not send extra traffic merely to populate a row.

Apache logs and bounded reads

Target / approval: APP1 Proxmox console, approved log reader; sudo only for protected files within J scope. Apache access/error files are one application source, alongside DC1 and HP's two other sources.

Read only the relevant interval. The conditional Ubuntu/Apache site uses the following paths; compare the active **ErrorLog** and **CustomLog** in **/etc/apache2/sites-available/labapp.conf** and **/etc/apache2/envvars** if they differ.

READ-ONLY - APP1 console, sudo/root or approved log-reader access. Show the last 40 application records, which may contain private identities/addresses:

```
sudo tail -n 40 /var/log/apache2/labapp-access.log
sudo tail -n 40 /var/log/apache2/labapp-error.log
```

Check: match the authenticated account, path, status, source and time. Expected role decisions are normally HTTP 200 allow and HTTP 403 authenticated denial. HTTP 401/wrong password is a different outcome. Compare the effective **/etc/labapp/groups**, **AuthGroupFile**, and relevant authorization/error record. Do not publish the password file or Authorization headers. See Apache's log reference.

Inspect **/etc/logrotate.d/apache2**, current file permissions/approved reader access, size/rotation and selected time range against the measured disk plan. An empty view can mean a rotated file, wrong site, inaccessible path or missing request; it does not prove absence. Use the already approved bounded interval from the correct rotated source if needed, preserving raw custody. Change retention/access only under its separately approved scope; never clear records.

READ-ONLY - APP1 local Linux console, approved log-reader; elevate only if journal access requires it. Optional service troubleshooting since local midnight; application request evidence remains in Apache's files:

```
journalctl --since today --no-pager
```

Check: match timezone/service messages to the action. Resolve permission/time-filter gaps before generating another event. This command does not start a log collector; journalctl documents the filters.

Source and observer time comparison

Use an independent observer clock on the approved existing lab administrator station. UTC means Coordinated Universal Time. Preserve both the source's original timestamp/offset and its UTC conversion. Record the observer's UTC at the action, the source event UTC and the signed difference **source UTC minus observer UTC**, in seconds; note observer resolution and manual timing/rounding. This difference is not a measured ingestion latency or guaranteed clock skew.

- DC1: use event XML **TimeCreated SystemTime** in UTC, alongside its displayed local time/timezone.
- Apache: preserve the access line's timestamp and numeric offset; convert to UTC before comparing. Check the error log's selected format/timezone separately.
- HP: inspect the selected release's displayed log timezone and firewall clock; convert using its actual offset. Do not assume browser time equals source time.

READ-ONLY - approved Windows observer, local PowerShell, standard user. Capture current observer UTC without changing time or accessing another computer:

```
(Get-Date).ToUniversalTime().ToString('o')
```

Check: retain actual times with the relevant event/action and private evidence. Inspect Windows **Date & time** and APP1's lab-only time views plus HP's clock/settings. If the difference prevents correlation, record the gap and obtain the approved time correction; never change directory time, invent timestamps or rerun failures to hide it.

Five benign event procedures

Targets / approval: existing WIN1/DC1/APP1 and physical HP, J plus the exact relevant active-test scope. Freeze one action per required category; reuse current prior evidence when it has the same state, full context and all required observations. Five identical requests do not count.

1. **E01 - Directory authentication success:** confirm WIN1 is connected to DC1, the account is a standard synthetic lab user and the known correct Windows credential works. Sign out/in once within approval. On DC1 find the attributable successful Kerberos record, normally 4768 with success result, using account/client/action time. A cached offline logon does not qualify.
2. **E02 - Directory authentication failure:** inspect effective lockout policy and the selected synthetic account's current failure/lockout margin with the directory administrator. Proceed only when one failure is safe. Make **one deliberately wrong-password attempt**, then stop even if no record appears. Find its matching DC1 failure, often 4771; inspect the actual failure code. Never guess repeatedly, disable lockout or use an administrator/production identity.
3. **E03 - Application access allowed:** on WIN1 use a clean browser context, valid distinct **ava application** credentials and **https://app.ad.lab.test/reports/**. Obtain a server request/HTTP 200 and corresponding Apache account/path record. Reuse the Phase 3 case if it remains current.

4. **E04 - Application access denied:** with valid ava app credentials request **https://app.ad.lab.test/maintenance/** in a fresh server request. Expect authenticated HTTP 403; compare the access/error record with ava absent from **LabMaintainers**. Distinguish 401, name/TLS failure or an offline service.
5. **E05 - Firewall access denied:** use one already approved known-listening lab-only deny case, such as DC1-to-APP1 TCP 443 from the network checks. First retain WIN1's successful HTTPS control and APP1's listener observation. Make the single frozen request; locate its OPNsense block with the actual policy/rule evidence. No home/production target or new listener/permit is authorized here. If that safe scope/control is unavailable, leave E05 NOT TESTED.

Record E01-E05 in **Deliverable 03 > Security Events**: actual source/action, synthetic context, source UTC, observer UTC, difference, concise event summary, expected/actual result and private evidence. Record reader/retention/clock observations in its notes or linked private evidence record. Link **Evidence Index** and **Test Cases**, FWT-07.01-.07/.09; include operator/date/version and separate action/policy observation. Approved two-source alternatives must still expose all five categories; do not silently omit directory or firewall evidence.

Denied-access investigation

Targets / approval: the already recorded E04 or E05, its request view, effective policy and source logs; J scope. No extra attack exercise is needed.

1. In **Deliverable 03 > Denied Access Investigation**, reference the Security Events ID. Explain who requested what: requester/account, client/machine context, resource/service, source and destination, actual time and visible outcome.
2. For E04 inspect valid authentication, request status, effective **AuthGroupFile/Require group**, **/etc/labapp/groups** and the correlated Apache records. For E05 inspect exact ingress/source/destination/service, rule/log identifier, aliases/order/states and the known listener. Record the intended enforcing group/rule and its actual event-time evidence.
3. Link one working allowed control to that same service and a second observation. For E04, use the matching C02 denial and C04 allowed control from before revocation when they share the event-time policy and service state. For E05, use the matched WIN1 or administrator listener control. Keep revoked permissions in place and leave the finding unverified if neither event has an applicable control.
4. Explain which evidence distinguishes a correct policy denial from incorrect credentials, broken DNS, stopped service or network failure. A 401 is not a valid-user role denial; a timeout alone does not prove a firewall control. Where multiple layers could deny, attribute only what the matching observation establishes.
5. Record conclusion, limitations, policy/log/control/second-observation evidence IDs and any defect. Link FWT-07.08/.09 in **Test Cases** and actual corrections/retests in **04 > Issue Log**, preserving the first failure. Missing evidence means unverified/NOT TESTED, not an inferred PASS.

Capacity target and private path review

Target / approval: WIN1 local Windows PowerShell 5.1, standard user with approved private write access; explicit L automation scope. Exactly one operation reads fixed-disk capacity and replaces **WIN1-capacity.json**. No remote execution, scheduled jobs or additional automation is included.

Inspect **System > About** / computer-domain properties: computer **WIN1**, domain **ad.lab.test**. Confirm the same identity with the script's local **Win32_ComputerSystem** query. Review the existing script and output directory outside every Git checkout, its owner/access rights, existing contents and target file. Keep the script/output/private inventory outside Git.

Check the directory, every parent and existing summary for junctions/symlinks/reparse points or another redirection. The existing guard walks parent paths for **.git**, and rejects missing/non-directory paths; it is not a canonical-path, ACL or reparse-point security boundary. Reject a root, shared/unreviewed directory or redirected path manually. Approve replacement of an existing summary and preserve required prior evidence privately first. This task does not rewrite or extend the reviewed guard.

Set **$LabCapacityScriptPath** and **$LabCapacityOutputDirectory** privately to the exact reviewed absolute file/directory paths in the current WIN1 shell. No credential belongs in an argument. Use the existing approved signing/execution-policy path; no execution-policy bypass or convenience elevation.

Complete WIN1 capacity script

Save this complete existing script unchanged in the approved private file. Get-CimInstance without a remote target uses the local computer. The fixed-disk filter and one output filename define the operation's scope.

CONFIGURATION CHANGE - WIN1 local Windows PowerShell, standard user with approved directory write rights. The script reads local fixed-disk capacity and writes only its reviewed summary; OutputDirectory is the single required runtime path:

```
param([Parameter(Mandatory)][string]$OutputDirectory)
$system = Get-CimInstance -ClassName Win32_ComputerSystem
if ($system.Name -ne 'WIN1' -or $system.Domain -ne 'ad.lab.test') {
    throw 'Target is outside the approved lab operation.'
}
$directory = Get-Item -LiteralPath $OutputDirectory -ErrorAction Stop
if (-not $directory.PSIsContainer) { throw 'Output must be a directory.' }
$cursor = $directory
while ($null -ne $cursor) {
    if (Test-Path -LiteralPath (Join-Path $cursor.FullName '.git')) {
        throw 'Keep the capacity record outside Git.'
    }
    $cursor = $cursor.Parent
}
$capacity = Get-CimInstance -ClassName Win32_LogicalDisk -Filter 'DriveType=3' |
    Select-Object DeviceID, Size, FreeSpace
$capacity | ConvertTo-Json |
    Set-Content -LiteralPath (Join-Path $directory.FullName 'WIN1-capacity.json') -Encoding utf8
```

Check before running: target/domain, exact private path, other-file preservation and one replacement filename match approval. Save the before-state and intended output hash/mtime if it exists. Stop if identity/path is uncertain; do not modify the real script to make it run elsewhere.

Successful and repeated execution

CONFIGURATION CHANGE - WIN1, same standard-user local PowerShell and privately assigned approved paths. Invoke this one reviewed script; do not supply another computer or run it remotely:

```
& $LabCapacityScriptPath -OutputDirectory $LabCapacityOutputDirectory
```

1. Confirm successful exit, then open the one **WIN1-capacity.json** locally. Check **DeviceID**, **Size**, **FreeSpace**, in bytes, for each fixed drive.
2. Independently open **File Explorer > This PC > drive Properties**. Compare displayed byte counts with the output, converting consistently if needed: one GiB is **1,073,741,824 bytes**. Record observation times; free space can change naturally between views.
3. Run the same command a second time successfully. Compare directory contents, other-file hashes/mtimes and before/after system/account/network settings: only the intended summary is replaced. A successful exit or JSON parse alone does not establish correct capacity or no side effects.

Record **03 > Automation Verification**, A01 approval/usefulness, A02 script/identity/path review, A03 first output/OS comparison and A04 repeat; link FWT-08.01-.04 and private evidence. Do not publish capacity output or broad machine inventory.

Invalid-target rejection

Target / approval: WIN1, standard user, same private output directory and L-approved negative test. Save the real script's hash/mtime and current summary's hash/mtime privately.

1. Make the approved disposable private copy; change only expected hostname **'WIN1'** to **'WIN1-INVALID-TEST'**. Verify actual WIN1; preserve the real script and all other code. Review/sign the copy under existing approved policy; a signing-policy block does not prove target-guard rejection.
2. Set **$LabCapacityInvalidTargetScriptPath** privately to that reviewed disposable copy. Invoke once; the target guard must reject before the disk query/output write. Never rename WIN1, change its domain or contact another host.

CONFIGURATION CHANGE attempt - WIN1 local PowerShell, standard user; approved disposable test copy only. Expected safe rejection, not a successful automation run:

```
& $LabCapacityInvalidTargetScriptPath -OutputDirectory $LabCapacityOutputDirectory
```

Check: Target is outside the approved lab operation. is observed. Independently inspect the existing summary's unchanged hash/mtime and the real script's unchanged content, plus no new output. Record actual error/no-write evidence in **03 > Automation Verification**, A05 / FWT-08.05. Remove only the approved disposable copy during cleanup. A changed real target allowlist is a failure, not this test.

Invalid-output-path rejection

Target / approval: WIN1, real unchanged script, L-approved safe negative path. Choose a reviewed known nonexistent directory or an existing ordinary private file as the argument; do not create a directory, point at a Git repository or write into a protected location for this test.

Set **$LabCapacityInvalidOutputPath** privately to that exact invalid path. Preserve the successful summary's hash/mtime before the attempt.

CONFIGURATION CHANGE attempt - WIN1 local PowerShell, standard user, real script and approved invalid argument. Expected Get-Item/non-directory rejection with no summary write:

```
& $LabCapacityScriptPath -OutputDirectory $LabCapacityInvalidOutputPath
```

Check: record the actual **Get-Item** error for a nonexistent path or **Output must be a directory.** for an ordinary file. Inspect the intended successful summary independently after this rejection too: its hash/mtime remains unchanged, no new file appears and the ordinary file, if used, is unchanged. Record A06 / FWT-08.06. Record all before/after settings, other-file preservation and publication review at A07 / FWT-08.07. Both target and path rejections are mandatory; one cannot substitute for the other.

Safe evidence and recovery

Use **03 > Evidence Index** for custody of private originals and separately reviewed public summaries. Preserve source event/log records, observer times, exact policy snapshot, script review/output/errors and failed attempts. Public evidence excludes raw logs, actual addresses, credentials, keys, password hashes and filled working inventories. **04 > Issue Log** owns observed defects, approved correction and later retest; owner acceptance remains separate.

On a log gap inspect approved auditing, rights, source paths/time/rotation and rule logging before another action; never clear logs or repeat failed passwords. On script failure stop and inspect identity, literal path, rights, units and error. Do not elevate, widen paths or bypass policy. Cleanup can remove only the approved output/test copy, after preserving evidence, and must preserve every other file/system setting. No extra automation or log-clearing script is introduced. Missing mandatory evidence stays NOT TESTED.

Phase 5 mock and connection tools

Use these tools for the guide's network tests (FWR/FWT-01/02/06). Run the authorized overlay mock during Phase 3 remote access tests, before revocation. Keep its matched control, denial and cleanup evidence for Phase 5.

Purpose and location: under the applicable D/F/H/I/K scopes, use existing lab guests to distinguish policy denial from a broken service. Freeze source, target, identity, single destination port, expected result, observer, stop condition and cleanup before each FWT-01/FWT-02/FWT-06 check.

1. Establish a working allowed control: WIN1 opens APP1 HTTPS and the dedicated administrator opens the lab management interfaces. Compare the active listener to the relevant permit/policy. If the control fails, diagnose it before the negative test.
2. For FWT-01's protected-boundary mock, use **APP1's lab address**, never a real protected address. Under the approved temporary rule scope, add a separate **MOCK_PROTECTED** alias containing that lab address and a logged Corporate rule blocking **WIN1 > MOCK_PROTECTED > TCP 443**, ahead of FF-03. Keep real protected networks physically absent. Make the one frozen request, compare its denial with this exact rule/counter, remove only the temporary mock rule/alias, and confirm normal FF-03 access again. The real isolation conclusion also requires cable/routes/trust review; the mock is one behavioral observation.
3. For FWT-02 unlisted inter-zone access, use DC1 as the source to the known listening APP1 TCP 443 while only WIN1 has FF-03 permission. For ordinary-user management denial, use WIN1 to Proxmox TCP 8006 with the dedicated-admin control working. Compare the intended network/host deny, not just a failed password prompt.
4. For FWT-06's wrong listening port, temporarily configure Apache's existing APP1 service to listen on one approved unused mock port such as **8443**. This example is conditional on that exact port/scope approval. Save the site/ports files and add the separate temporary file below. Use the Apache verification procedure for service and listener checks.

CONFIGURATION CHANGE - APP1 root editor, temporary /etc/apache2/conf-available/lab-mock.conf; approved 8443 mock only. Configuration content, not commands; reuse the synthetic document tree and deny page content. The positive control needs only a listening TCP connection:

```
Listen 8443
<VirtualHost *:8443>
    ServerName app.ad.lab.test
    DocumentRoot "/srv/labapp"
    <Directory "/srv/labapp">
        Require all denied
    </Directory>
</VirtualHost>
```

Check before enabling: the port is unused, the exact temporary listener has approval, and no new Tailscale grant or ordinary-user permit is added.

CONFIGURATION CHANGE followed by READ-ONLY - APP1 console, sudo/root, temporary mock scope. Enable only that named configuration. Run each line separately. Reload only after **Syntax OK**:

```
sudo a2enconf lab-mock
sudo apachectl configtest
sudo systemctl reload apache2
sudo ss -lntp
```

Check: proceed to reload only after Syntax OK. Confirm Apache listens on 8443; if parsing fails, disable the named mock configuration and restore the saved files before reloading.

5. Use the APP1 console replacement transaction to add **ip saddr LAB_ADMIN_IP tcp dport 8443 counter accept** after the physical-source HTTPS permit and before the input chain's closing brace. Retain the tailscale0 drop and final drop policy. Replace the source with the exact dedicated administrator IPv4 address. OPNsense's corresponding temporary FF-05 Management permit must name the same source/destination/port. Keep the vendor TCP-443-only grant unchanged. Confirm the administrator reaches the listener, then make the frozen remote wrong-port request. Match the denial to the actual vendor/host rule state or counter before attributing it.
6. After the recorded check, disable only **lab-mock** with the Ubuntu **a2disconf lab-mock** service helper, syntax-check, reload and confirm 8443 is no longer listening. Remove the exact temporary host/firewall permits through the saved transaction. Recheck production/home isolation and normal APP1 HTTPS. Preserve the temporary-rule evidence privately; add no permanent VM or zone.

READ-ONLY bounded connection check - approved Windows lab source, local PowerShell, standard user. Set **$LabApprovedTarget** privately to the one approved lab-owned address/name and **$LabApprovedPort** to its single frozen port. This makes a TCP connection attempt only; it is not a scan or a protected-network probe:

```
$connection = [System.Net.Sockets.TcpClient]::new()
try {
    $attempt = $connection.ConnectAsync($LabApprovedTarget, $LabApprovedPort)
    if (-not $attempt.Wait(5000)) { throw 'Lab connection timed out.' }
    'TCP CONNECTED'
} catch {
    'NO TCP CONNECTION: ' + $_.Exception.Message
} finally {
    $connection.Dispose()
}
```

Check: the positive control prints TCP CONNECTED; a denied attempt prints NO TCP CONNECTION and has matching intended policy/log/counter evidence. A timeout alone remains inconclusive. Do not supply a home/production address, loop through ports or rerun repeatedly to force a result.

If it fails: preserve the observation, restore only the named temporary configuration under the approved plan and recheck normal access. Missing approval or working listener means NOT TESTED, not permission to broaden access.